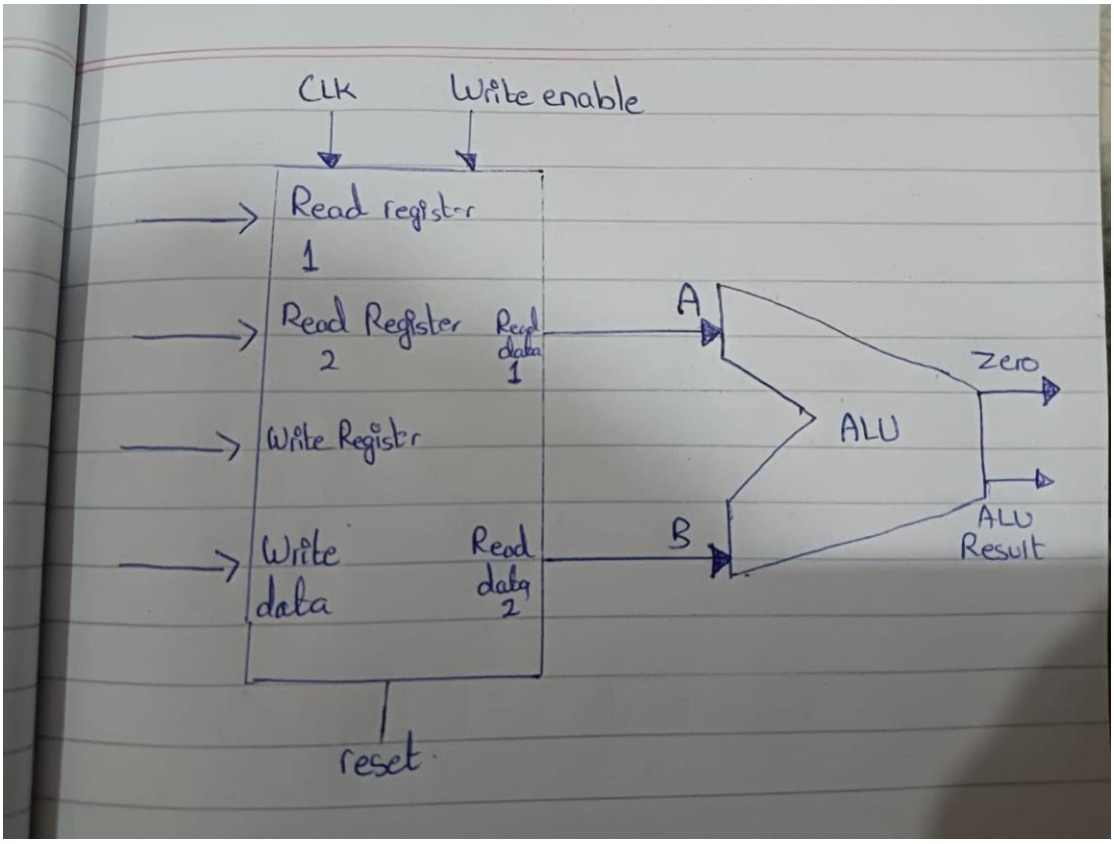
Lab 07 – Worksheet

Name: Mohammad Husain Naqvi Syed Fakhar Iqbal Rizvi	ID: 11062 11091	Section: T-4

Note: Assumptions and logics should be explained separately in tasks after the task results.

Task 1

Block Diagrams



Comments

Task 2

Modules + Testbench:

Register File:

```
`timescale 1ns / 1ps
module RegisterFile(
    input wire clk,
    input wire rst,
    input wire WriteEnable,
    input wire [4:0] rs1,
    input wire [4:0] rs2,
    input wire [4:0] rd,
    input wire [31:0] WriteData,
    output reg [31:0] ReadData1,
    output reg [31:0] ReadData2
);
    reg [31:0] regs [31:0];
    integer i;
    initial begin
        for (i = 0; i < 32; i = i + 1) begin
            regs[i] = i;
        end
    end
    always @(*) begin
        ReadData1 = regs[rs1];
        ReadData2 = regs[rs2];
    end
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            for (i = 0; i < 32; i = i + 1) begin
                regs[i] <= i;
            end
        end else if (WriteEnable && (rd != 5'b00000)) begin
            regs[rd] <= WriteData;
        end
    end

endmodule
```

Top ALU_RegisterFile Module:

```
`timescale 1ns / 1ps
module top_rf_alu(
    input wire clk,
    input wire btnC,            // Reset button
    input wire [15:0] sw,       // Switches
    output wire [15:0] led,     // LEDs
    output wire [6:0] seg,      // Seven segment display
    output wire [3:0] an        // Seven segment anode
);
    wire rst_db;
```

```
    debouncer db_rst (
        .clk(clk),
        .pbin(btnC),
        .pbout(rst_db)
    );
    wire [31:0] sw_readData;
    wire [15:0] sw_led_mirror;
    switches sw_if (
        .clk(clk),
        .rst(rst_db),
        .writeData(32'b0),
        .writeEnable(1'b0),
        .readEnable(1'b1),       //we always read the switches
        .memAddress(30'b0),
        .sw(sw),
        .readData(sw_readData),
        .leds(sw_led_mirror)
    );

// Switch 0 controls writing to the register file
    wire WriteEnable = sw_readData[0];

    // sw[4:1] (4 bits) selects the second source register rs2 (x0 to x15)
    wire [4:0] rs2 = {1'b0, sw_readData[4:1]};

    // sw[9:5] (5 bits) selects the first source register rs1 (x0 to x31)
    wire [4:0] rs1 = sw_readData[9:5];

    // sw[10] can be left as an extra bit or used for write data
    wire [31:0] WriteData = {31'b0, sw_readData[10]};

    // sw[14:11] (4 bits) selects the ALU operation
    wire [3:0] ALUControl = sw_readData[14:11];

    // Target register rd can match rs2 (or keep as 5'd2)
    wire [4:0] rd = rs2;        //rd=x2
    wire [31:0] ReadData1;      //output 1
    wire [31:0] ReadData2;      //output 2
    wire [31:0] alu_result;
    wire alu_zero;
    //Register File
    RegisterFile rf (
        .clk(clk),
        .rst(rst_db),
        .WriteEnable(WriteEnable),
        .rs1(rs1),
        .rs2(rs2),
        .rd(rd),
        .WriteData(WriteData),
        .ReadData1(ReadData1),
        .ReadData2(ReadData2)
    );
    // ALU FILE
    ALU_32bit alu (
        .A(ReadData1),
        .B(ReadData2),
        .ALUControl(ALUControl),
```

```
        .Result(alu_result),
        .Zero(alu_zero)
    );
    assign led[14:0] = alu_result[14:0];
    assign led[15] = alu_zero;

    leds led_if (
        .clk(clk),
        .rst(rst_db),
        .btns(16'b0),
        .writeData(alu_result),
        .writeEnable(1'b0),
        .readEnable(1'b0),
        .memAddress(30'b0),
        .display_value(alu_result[3:0]),
        .readData(),
        .seg(seg),              // Seven-segment display output
        .an(an)                 // Seven-segment anode control
    );

endmodule
```

Debouncer:

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 14.10.2026 11:04:22
// Design Name:
// Module Name: debouncer
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


`timescale 1ns / 1ps

module debouncer (
    input wire clk,
    input wire pbin,
    output reg pbout
);
```

```
    // 16-bit counter creates ~0.65ms debounce window at 100MHz clock
    reg [15:0] counter = 16'd0;
    reg state = 1'b0;

    always @(posedge clk) begin
        if (pbin != state) begin
            counter <= counter + 1'b1;
            if (counter == 16'hFFFF) begin
                state <= pbin;
                counter <= 16'd0;
            end
        end else begin
            counter <= 16'd0;
        end
        pbout <= state;
    end

endmodule
```

Swtiches:

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 14.10.2026 11:04:22
// Design Name:
// Module Name: debouncer
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


`timescale 1ns / 1ps

module debouncer (
    input wire clk,
    input wire pbin,
    output reg pbout
);

    // 16-bit counter creates ~0.65ms debounce window at 100MHz clock
    reg [15:0] counter = 16'd0;
    reg state = 1'b0;
```

```
    always @(posedge clk) begin
        if (pbin != state) begin
            counter <= counter + 1'b1;
            if (counter == 16'hFFFF) begin
                state <= pbin;
                counter <= 16'd0;
            end
        end else begin
            counter <= 16'd0;
        end
        pbout <= state;
    end

endmodule
```

Alu_32bit:

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 14.10.2026 11:05:30
// Design Name:
// Module Name: Alu_32bit
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


`timescale 1ns / 1ps

module ALU_32bit(
    input wire [31:0] A,
    input wire [31:0] B,
    input wire [3:0] ALUControl,
    output reg [31:0] Result,
    output wire Zero
);

    always @(*) begin
        case (ALUControl)
            4'b0010: Result = A + B;            // ADD 2 + 5 = 7
            4'b0110: Result = A - B;            // SUB 2-5= 3 (unsigned)
            4'b0000: Result = A & B;            // AND 00101  & 0010 = 0
            4'b0001: Result = A | B;            // OR  (1)
            4'b1100: Result = A ^ B;            // XOR (12)
            4'b0100: Result = A << B[4:0];      // SLL (4)
```

```
            4'b0101: Result = A >> B[4:0];       // SRL (5)
            default: Result = 32'd0;
        endcase
    end

    // Zero flag is high when Result is exactly 0
    assign Zero = (Result == 32'd0);

endmodule
```

LED:

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 14.10.2026 11:05:30
// Design Name:
// Module Name: Alu_32bit
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


`timescale 1ns / 1ps

module ALU_32bit(
    input wire [31:0] A,
    input wire [31:0] B,
    input wire [3:0] ALUControl,
    output reg [31:0] Result,
    output wire Zero
);

    always @(*) begin
        case (ALUControl)
            4'b0010: Result = A + B;            // ADD 2 + 5 = 7
            4'b0110: Result = A - B;            // SUB 2-5= 3 (unsigned)
            4'b0000: Result = A & B;            // AND 00101  & 0010 = 0
            4'b0001: Result = A | B;            // OR  (1)
            4'b1100: Result = A ^ B;            // XOR (12)
            4'b0100: Result = A << B[4:0];      // SLL (4)
            4'b0101: Result = A >> B[4:0];      // SRL (5)
            default: Result = 32'd0;
        endcase
```

```
    end

    // Zero flag is high when Result is exactly 0
    assign Zero = (Result == 32'd0);

endmodule
```

TESTBENCH:
tb_top_rf_alu:

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 14.10.2026 11:13:37
// Design Name:
// Module Name: Top_Alu_Rf_tb
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


`timescale 1ns/1ps

module tb_top_rf_alu;
    reg clk;
    reg rst;
    reg WriteEnable;
    reg [4:0] rs1, rs2, rd;
    reg [31:0] WriteData;
    wire [31:0] ReadData1, ReadData2;
    reg [3:0] ALUControl;
    wire [31:0] ALUResult;
    wire Zero;

    RegisterFile rf (
        .clk(clk),
        .rst(rst),
        .WriteEnable(WriteEnable),
        .rs1(rs1),
        .rs2(rs2),
        .rd(rd),
        .WriteData(WriteData),
        .ReadData1(ReadData1),
        .ReadData2(ReadData2)
    );
```

```
ALU_32bit alu (
    .A(ReadData1),
    .B(ReadData2),
    .ALUControl(ALUControl),
    .Result(ALUResult),
    .Zero(Zero)
);

// Clock generator: 100MHz (10ns period)
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

localparam IDLE           = 3'd0;
localparam WRITE_REGS     = 3'd1;
localparam READ_REGISTERS  = 3'd2;
localparam ALU_OPERATION   = 3'd3;
localparam WRITE_REGISTERS = 3'd4;

reg [2:0] state;
reg [1:0] init_idx;
reg [3:0] op;
reg [4:0] dest;

initial begin
    rst = 1;
    @(posedge clk);
    @(posedge clk);
    rst = 0;
    repeat (120) @(posedge clk);
    $finish;
end

always @(posedge clk or posedge rst) begin
    if (rst) begin
        state <= IDLE;
        WriteEnable <= 0;
        rs1 <= 0; rs2 <= 0; rd <= 0;
        WriteData <= 0;
        ALUControl <= 4'b0010;
        init_idx <= 0;
        op <= 0;
        dest <= 0;
    end else begin
        case (state)
            IDLE: begin
                WriteEnable <= 0;
                init_idx <= 0;
                op <= 0;
                state <= WRITE_REGS;
            end

            WRITE_REGS: begin
                WriteEnable <= 1;
                case (init_idx)
```

```
            2'd0: begin
               rd <= 5'd1;
               WriteData <= 32'h1010_1010;
               init_idx <= 1;
            end
            2'd1: begin
               rd <= 5'd2;
               WriteData <= 32'h0101_0101;
               init_idx <= 2;
            end
            2'd2: begin
               rd <= 5'd3;
               WriteData <= 32'h0000_0005;
               init_idx <= 0;
               WriteEnable <= 0;
               state <= READ_REGISTERS;
            end
         endcase
      end

      READ_REGISTERS: begin
         WriteEnable <= 0;
         rs1 <= 5'd1;
         rs2 <= 5'd2;
         case (op)
            4'd0: begin ALUControl <= 4'b0010; dest <= 5'd4;  end // ADD
            4'd1: begin ALUControl <= 4'b0110; dest <= 5'd5;  end // SUB
            4'd2: begin ALUControl <= 4'b0000; dest <= 5'd6;  end // AND
            4'd3: begin ALUControl <= 4'b0001; dest <= 5'd7;  end // OR
            4'd4: begin ALUControl <= 4'b1100; dest <= 5'd8;  end // XOR
            4'd5: begin ALUControl <= 4'b0100; dest <= 5'd9;  end // SLL
            4'd6: begin ALUControl <= 4'b0101; dest <= 5'd10; end // SRL
            4'd7: begin
               rs1 <= 5'd1;
               rs2 <= 5'd1;
               ALUControl <= 4'b0110; // x1 - x1 => 0
               dest <= 5'd11;
            end
            4'd8: begin
               ALUControl <= 4'b0010;
               dest <= 5'd12;
            end
            4'd9: begin
               rs1 <= 5'd12;
               rs2 <= 5'd0;
               ALUControl <= 4'b0010;
               dest <= 5'd0;          // Test write to x0 (should be discarded)
            end
         endcase
         state <= ALU_OPERATION;
      end

      ALU_OPERATION: begin
         state <= WRITE_REGISTERS;
      end

      WRITE_REGISTERS: begin
```

```
                case (op)
                    4'd0, 4'd1, 4'd2, 4'd3, 4'd4, 4'd5, 4'd6: begin
                        rd <= dest;
                        WriteData <= ALUResult;
                        WriteEnable <= 1;
                        op <= op + 1;
                        state <= READ_REGISTERS;
                    end
                    4'd7: begin
                        rd <= dest; // x11
                        WriteData <= (Zero ? 32'd1 : 32'd0);
                        WriteEnable <= 1;
                        op <= 8;
                        state <= READ_REGISTERS;
                    end
                    4'd8: begin
                        rd <= 5'd12;
                        WriteData <= 32'hAAAA_5555;
                        WriteEnable <= 1;
                        op <= 9;
                        state <= READ_REGISTERS;
                    end
                    4'd9: begin
                        WriteEnable <= 0;
                        rs1 <= 5'd4; // read x4 at the end
                        rs2 <= 5'd5; // read x5 at the end
                        state <= IDLE;
                    end
                endcase
            end
        endcase
    end
end

endmodule
```

REGISTER TB:

```
`timescale 1ns / 1ps

module tb_RegisterFile;

    // DUT Inputs
    reg clk;
    reg rst;
    reg WriteEnable;
    reg [4:0] rs1;
    reg [4:0] rs2;
    reg [4:0] rd;
    reg [31:0] WriteData;

    // DUT Outputs
    wire [31:0] ReadData1;
    wire [31:0] ReadData2;
```

```
    // Error counter
    integer errors = 0;

    // Instantiate the Device Under Test (DUT)
    RegisterFile dut (
        .clk(clk),
        .rst(rst),
        .WriteEnable(WriteEnable),
        .rs1(rs1),
        .rs2(rs2),
        .rd(rd),
        .WriteData(WriteData),
        .ReadData1(ReadData1),
        .ReadData2(ReadData2)
    );

    // 100 MHz Clock Generation (10ns period)
    initial begin
        clk = 1'b0;
        forever #5 clk = ~clk;
    end

    // Verification Task
    task check_outputs(
        input [4:0] r1_addr, input [31:0] exp1,
        input [4:0] r2_addr, input [31:0] exp2,
        input [8*30:1] test_name
    );
        begin
            rs1 = r1_addr;
            rs2 = r2_addr;
            #1; // allow combinational read paths to settle
            if (ReadData1 !== exp1) begin
                $display("[FAIL] %0s | ReadData1 (reg %0d) = 0x%08h, Expected = 0x%08h", test_name,
r1_addr, ReadData1, exp1);
                errors = errors + 1;
            end else if (ReadData2 !== exp2) begin
                $display("[FAIL] %0s | ReadData2 (reg %0d) = 0x%08h, Expected = 0x%08h", test_name,
r2_addr, ReadData2, exp2);
                errors = errors + 1;
            end else begin
                $display("[PASS] %0s | reg[%0d]=0x%08h, reg[%0d]=0x%08h", test_name, r1_addr,
ReadData1, r2_addr, ReadData2);
            end
        end
    endtask

    // Main Test Sequence
    initial begin
        // 1. Initialize signals and assert Reset
        rst = 1'b1;
        WriteEnable = 1'b0;
        rs1 = 5'd0;
        rs2 = 5'd0;
        rd  = 5'd0;
        WriteData = 32'd0;
```

```
#20;
@(posedge clk);
rst = 1'b0;
#1;

// Test 1: Verify Initial / Reset Values (regs[i] == i)
check_outputs(5'd0, 32'd0, 5'd5, 32'd5, "Reset Values (x0 & x5)");
check_outputs(5'd10, 32'd10, 5'd31, 32'd31, "Reset Values (x10 & x31)");

// Test 2: Standard Write to Register x5
@(negedge clk);
rd = 5'd5;
WriteData = 32'h1234_5678;
WriteEnable = 1'b1;

@(negedge clk);
WriteEnable = 1'b0; // Deassert write enable
check_outputs(5'd5, 32'h1234_5678, 5'd0, 32'd0, "Write to x5");

// Test 3: Attempt writing to Register x0 (Must remain 0)
@(negedge clk);
rd = 5'd0;
WriteData = 32'hDEAD_BEEF;
WriteEnable = 1'b1;

@(negedge clk);
WriteEnable = 1'b0;
check_outputs(5'd0, 32'h0000_0000, 5'd5, 32'h1234_5678, "Write to x0 Ignored");

// Test 4: Simultaneous dual-port read after writing to x6
@(negedge clk);
rd = 5'd6;
WriteData = 32'h0000_00AA;
WriteEnable = 1'b1;

@(negedge clk);
WriteEnable = 1'b0;
check_outputs(5'd5, 32'h1234_5678, 5'd6, 32'h0000_00AA, "Dual Read x5 & x6");

// Test 5: Overwrite existing value in x5
@(negedge clk);
rd = 5'd5;
WriteData = 32'h0000_0055;
WriteEnable = 1'b1;

@(negedge clk);
WriteEnable = 1'b0;
check_outputs(5'd5, 32'h0000_0055, 5'd6, 32'h0000_00AA, "Overwrite x5");

// Test 6: Verify Reset restores regs[i] = i
@(negedge clk);
rst = 1'b1;
@(posedge clk);
#1;
rst = 1'b0;
#1;
```

```
        check_outputs(5'd5, 32'd5, 5'd6, 32'd6, "Reset Restores Defaults");

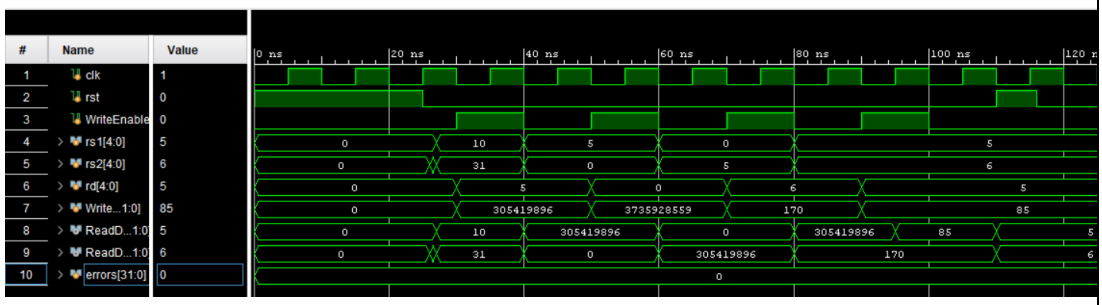
        // Final Report
        #20;
        $display("-----------------------------------------------");
        if (errors == 0)
            $display("ALL TESTS PASSED SUCCESSFULLY!");
        else
            $display("TEST FAILED WITH %0d ERRORS.", errors);
        $display("-----------------------------------------------");
        $finish;
    end

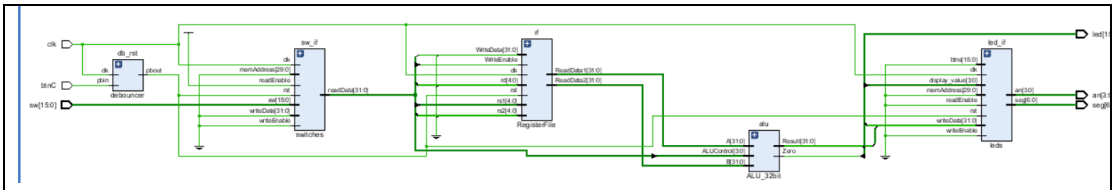
endmodule
```

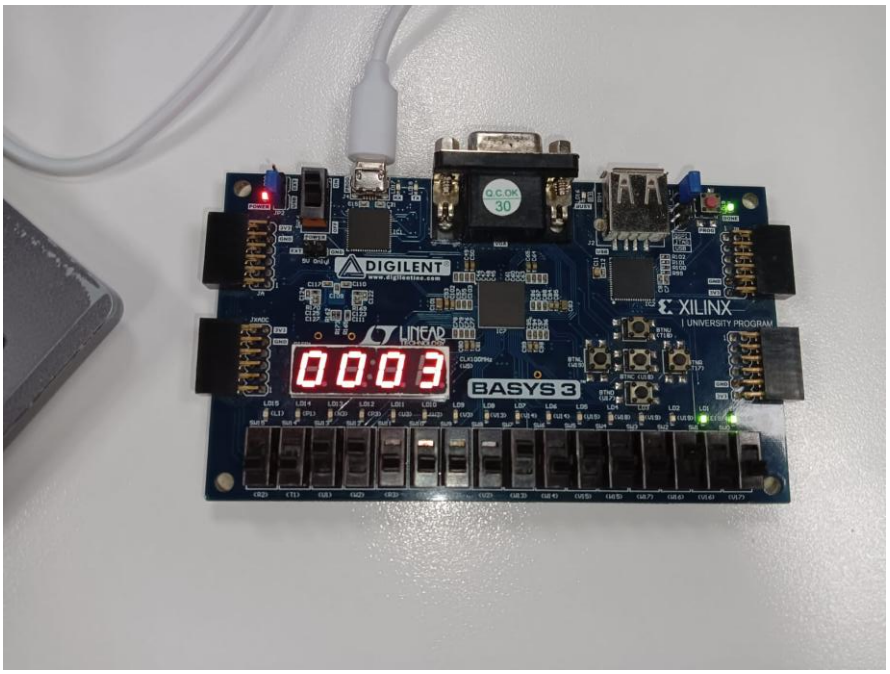
Results:

Task 3

RTL Schematics







Switches[4:1] = A = 0010= 2

Switches[8:5] = B = 0101

Switches[15:11] = ALU= 0110

A-B= 3

The Video of FPGA implementation has been uploaded on TEAMS

GITHUB links:

https://github.com/Fakhariqblal/caLAB_git_si11091-main

https://github.com/MohammadNaqvi1098/CALabs-Git-mn11062.git

Assessment Rubrics

Lab 7: Design and FPGA Implementation of the Register File for a Single-Cycle RISC-V Processor

Points Distribution

Task No.	LR2 Code	LR 5 Results	LR11 Design	AR7
Task 1	-	-	/10	/20
Task 2	/20	/10		
Task 3	/20	/20		
Total Points	/80			/20
CLO Mapped	CLO 1			

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3:Good Points 3	Level 4:Exemplary Points 4
LR2	**Program/Code / Simulation Model/ Network Model**	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	**Results & Plots**	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	**Report**	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.